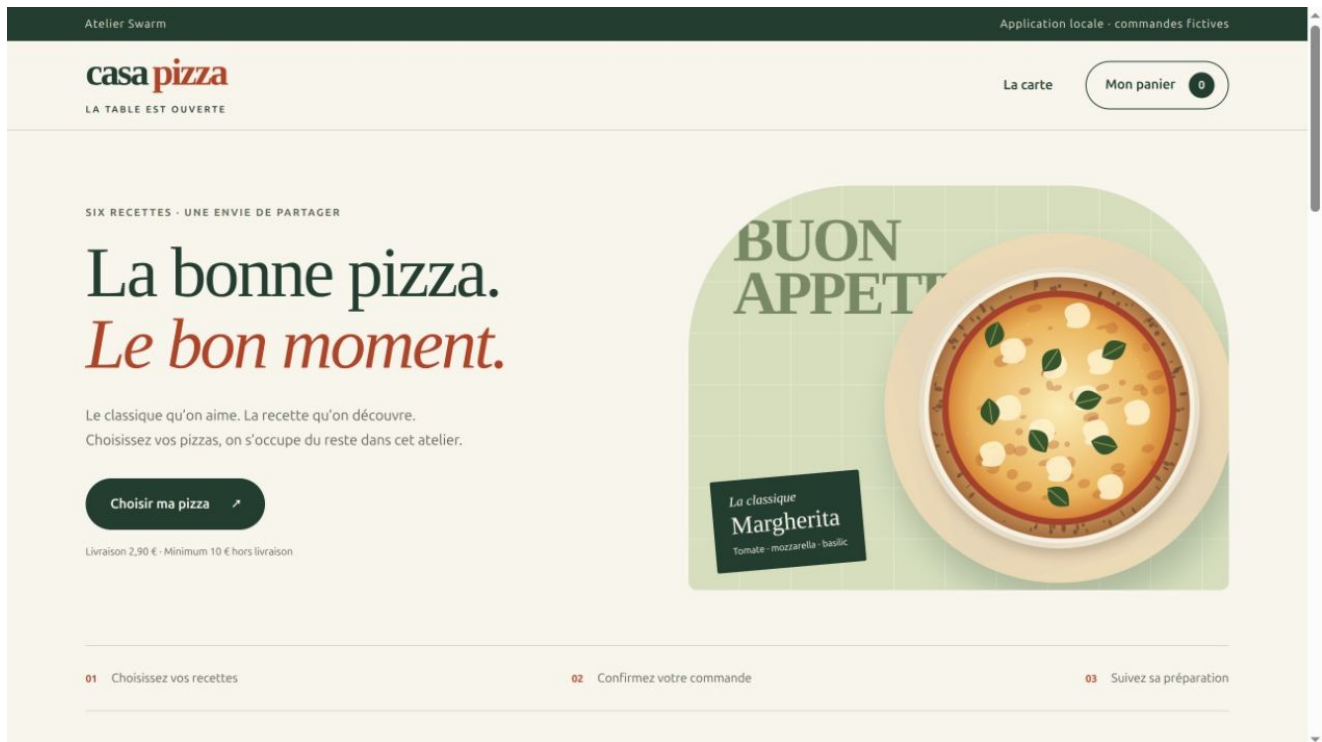


1 Swarm beginner training guide

Build and verify a pizza delivery application. A guided workshop to understand missions, agents, dependencies and human decisions.



Casa Pizza is a working local application with six recipes and persistent orders.

What you will achieve

You will prepare a requirement, launch an executor, read its activity, diagnose a failure and verify a delivery. You will then play the customer and restaurant roles to test a complete order.

For beginners. Edition dated 9 October 2026. Production screenshots show a workshop with one executor and trainer verification. The six-task plan and policy screenshots demonstrate configuration; those six tasks were not executed.

Commands use Linux and bash. The reference application remains French in both editions. New cockpit screenshots follow the guide language; historical screenshots retain their original labels.

2 Learning path

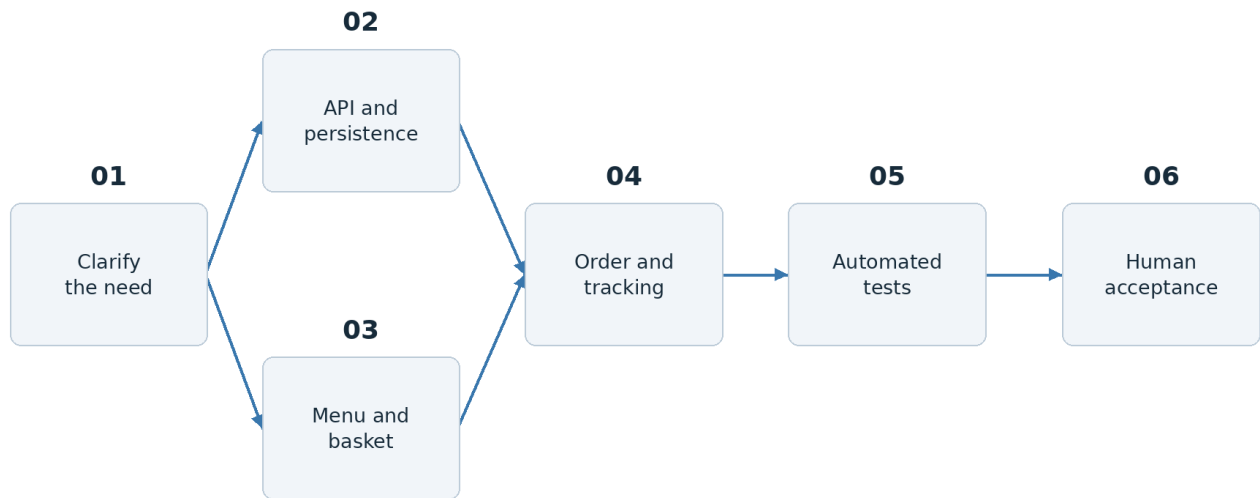
Follow the chapters in order for the complete workshop. For a specific problem, use the sections below.

- 03 What Swarm is for
- 04 Who does what
- 05 Prepare your computer
- 06 Try the local reference application
- 07 Create your Swarm project
- 08 Write a clear requirement
- 09 From preparation to execution
- 10 Read the guided mission
- 11 Read the task graph
- 12 Read a task detail
- 13 Explore and edit the graph
- 14 Launch a task with clear limits
- 15 Monitor an execution
- 16 Resolve a startup failure
- 17 Distinguish verification stages
- 18 Configure human review with checks
- 19 Define the test check
- 20 Review consequences before confirmation
- 21 Understand automatic validation
- 22 Check that evidence is current
- 23 Keep a human in the loop
- 24 Understand independent review
- 25 Give the design a direction
- 26 Review visual quality
- 27 Check the menu and basket
- 28 Test a useful error
- 29 Confirm and track an order
- 30 Update the restaurant status
- 31 Check final tracking and mobile use
- 32 Inspect a mobile confirmation
- 33 Verify before accepting
- 34 Diagnose Failed to fetch
- 35 Repeat and explain the journey
- 36 Workshop scope and evidence

3 What Swarm is for

Swarm is a local cockpit for organizing and managing work assigned to AI agents. It connects a requirement with tasks, execution attempts, evidence and an acceptance decision.

READING THE DEPENDENCIES



An arrow means the prerequisite must be accepted before the next task

Teaching diagram of the plan • This diagram is not execution evidence

This diagram explains a possible work structure. It does not show a successful autonomous execution.

A concrete example

For Casa Pizza, the requirement describes what the customer must be able to do. A task requests code and tests. The executor produces files. Swarm records activity and attempts; the trainer then tests the order journey and decides whether the criteria are met.

The cockpit exposes dependencies and waiting reasons. It can manage continuous starts and preauthorized checks when the corresponding organization and authorizations are configured. Displaying a graph is not enough to enable these features.

4 Who does what

Each actor has a different responsibility. Producing a result is separate from reviewing or accepting it.

Actor	Responsibility	In this workshop
User	Defines scope, authorizes, examines and decides	Trainer and learner
Planner	Organizes and delegates tasks	Hierarchy not configured
Executor	Uses tools to produce files and its report	One Codex executor
Swarm engine	Applies authorized conditions and checks	History and public operations
Independent reviewer	Examines a candidate in a separate session	Not used for the original verification

A chat AI and an executor

An API connection can help draft or analyze text. An executor with tools is needed to write project files. Configuring a chat connection does not establish that the AI can build the application.

Naming a task “Review” does not configure an independent reviewer. Check the actual role, session, authorization and evidence used.

5 Prepare your computer

Open a terminal. This is the window where you type the commands below, one line at a time. These commands are intended for Linux or a bash-compatible terminal.

Check the required programs

```
python3 --version
git --version
codex --version
```

Python 3.10 or later runs Casa Pizza and its tests. Git keeps a local history. Codex must already be installed and authenticated to generate code; the kit does not perform authentication.

Check Swarm

In the folder where you installed the Swarm repository, check for the swarm.sh launcher and the bin/swarm executable. The trainer should provide a working version before the workshop.

```
ls -l swarm.sh bin/swarm
./swarm.sh --help
./bin/swarm --help
```

Understand the folders

The Swarm folder contains the tool. The pizza-workshop folder contains the learner's work. The kit contains projet-pizza, the completed reference application. Do not generate code inside your usual application project.

Understand local addresses

127.0.0.1 refers to your own computer. The Swarm cockpit and the webapp use different ports. In the screenshots, Swarm uses 18842 and Casa Pizza uses 18841. The Swarm launcher normally uses 18792 unless you choose another port.

This workshop runs directly on your computer. If Swarm runs in a container, the trainer must first prepare the paths, agent programs, and access inside that container.

Expected result. All three programs respond, and you know where Swarm is installed.

6 Try the local reference application

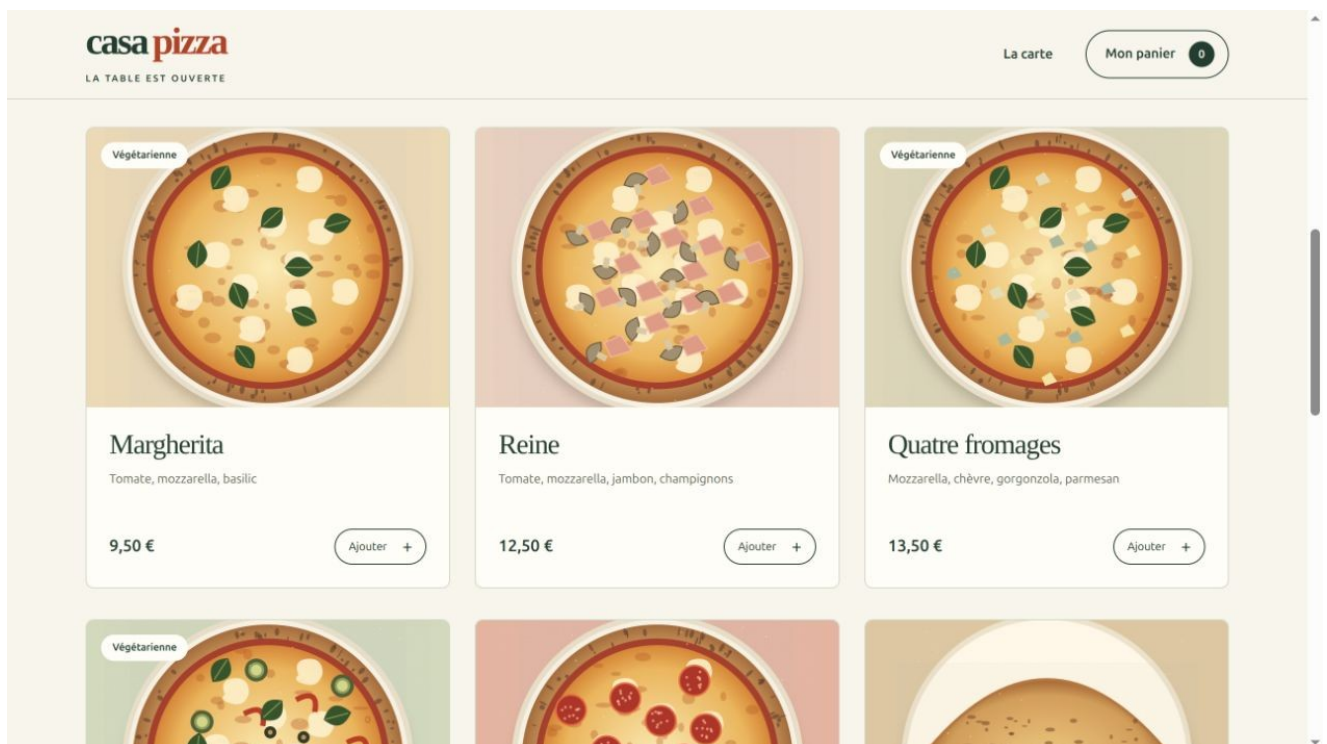
First, explore what a successful delivery should let you do. Extract the kit and open a terminal in the extracted folder.

1. Enter projet-pizza, the folder containing the reference code.
2. Start the server and keep this terminal open.
3. Open the client address below in your browser.

```
cd projet-pizza  
python3 app.py
```

Client address: <http://127.0.0.1:18841/>

Restaurant address: <http://127.0.0.1:18841/restaurant>



Ajouter means Add. These buttons add pizzas to the basket. All illustrations are local files.

The restaurant demo password is atelier when CASA_PIZZA_RESTAURANT_PASSWORD is not set. This default is only for the local workshop.

Expected result. The menu appears. To stop the server, return to its terminal and press Ctrl+C.

7 Create your Swarm project

This preparation creates a mission and a task in a fresh folder. It copies the requirements and working rules, initializes Git, and configures Codex. It does not launch an agent or copy any credentials.

1. Return to the extracted kit folder containing `atelier_swarm.py`.
2. Replace the executable path with the path to your installed Swarm executable.
3. Choose a new pizza-workshop folder. The script refuses an existing nonempty folder.

```
python3 atelier_swarm.py \  
  --swarm /path/to/swarm/bin/swarm \  
  --projet "$HOME/projects/pizza-workshop"
```

Replace `/path/to/swarm/bin/swarm` with your installation path. The `--projet` option means project. Quotes around the folder path preserve paths containing spaces. Keep the script's French option names exactly as shown.

Open the cockpit for this project

Go to your Swarm installation folder, then run:

```
./swarm.sh start --root "$HOME/projects/pizza-workshop"
```

Open the local address printed by the launcher. Do not share a session link containing private access. If another Swarm instance is running, choose a free port:

```
./swarm.sh start --root "$HOME/projects/pizza-workshop" \  
  --address 127.0.0.1:18842
```

Use the same address for status, open, and stop commands. Do not stop another project's mission to free its port.

Expected result. The terminal prints `Projet prêt`, meaning Project ready, and a mission identifier. Formation Casa Pizza is available in Swarm.

8 Write a clear requirement

Before sending an instruction, describe what the client and the restaurant should do. Observable behavior gives you a way to check the delivery.

► Préparation et méthode — reprendre un projet ou changer de méthode

Choisir un modèle de mission

Quel résultat souhaitez-vous obtenir ?

Aide : rédiger le besoin

Nom de la préparation

Casa Pizza pour un premier atelier

Votre besoin

Créer une webapp française de livraison de pizzas pour un atelier. Le client choisit parmi six pizzas, filtre les végétariennes, modifie son panier, renseigne une adresse fictive et confirme sa commande. Paiement à la livraison simulé. Le restaurant fait avancer le statut et le client suit sa commande. Les données persistent après redémarrage. Python 3 et SQLite, HTML CSS JS local, sans CDN ni package à installer. Vérifier le panier, les champs invalides, les montants recalculés par le serveur, les accès et le parcours mobile. Livrer code, tests, README et rapport. Aucun paiement réel, aucune adresse réelle et aucune publication Internet.]

Votre texte est enregistré dans ce projet. Aucun agent ne sera lancé.

Enregistrer le besoin Télécharger mon brouillon

Préparer un projet means **Prepare a project**. **Votre besoin** means **Your requirement**. **Enregistrer le besoin** saves the text without launching an agent.

1. Open Préparer un projet if you want to save your own draft.
2. Give it a name and enter the text below.
3. Click Enregistrer le besoin. Then return to the Formation Casa Pizza mission created by the kit.

Text to copy

Create a French pizza delivery webapp for a workshop. The client chooses from six pizzas, filters vegetarian options, changes the basket, enters a fictional address, and confirms the order. Simulate payment on delivery. The restaurant updates the status, and the client tracks the order. Data must survive a server restart. Use Python 3 and SQLite, with local HTML, CSS, and JavaScript; no CDN or packages to install. Check the basket, invalid fields, server-calculated totals, access controls, and the mobile journey. Deliver code, tests, a README, and a report. No real payment, real address, or Internet publication.

Expected result. The requirement is saved. The kit also supplies detailed requirements in docs/BESOIN.md. The script creates this workshop's mission; saving this draft does not automatically adopt it into that mission.

9 From preparation to execution

Preparation helps clarify the project. A saved proposal is different from adopted work or a launched agent.

► Préparation et méthode — reprendre un projet ou changer de méthode

Choisir un modèle de mission

Quel résultat souhaitez-vous obtenir ?

[Aide : rédiger le besoin](#)

Nom de la préparation

Casa Pizza pour un premier atelier

Votre besoin

Créer une webapp française de livraison de pizzas pour un atelier. Le client choisit parmi six pizzas, filtre les végétariennes, modifie son panier, renseigne une adresse fictive et confirme sa commande. Paiement à la livraison simulé. Le restaurant fait avancer le statut et le client suit sa commande. Les données persistent après redémarrage. Python 3 et SQLite, HTML CSS JS local, sans CDN ni package à installer. Vérifier le panier, les champs invalides, les montants recalculés par le serveur, les accès et le parcours mobile. Livrer code, tests, README et rapport. Aucun paiement réel, aucune adresse réelle et aucune publication Internet.]

Votre texte est enregistré dans ce projet. Aucun agent ne sera lancé.

Enregistrer le besoin Télécharger mon brouillon

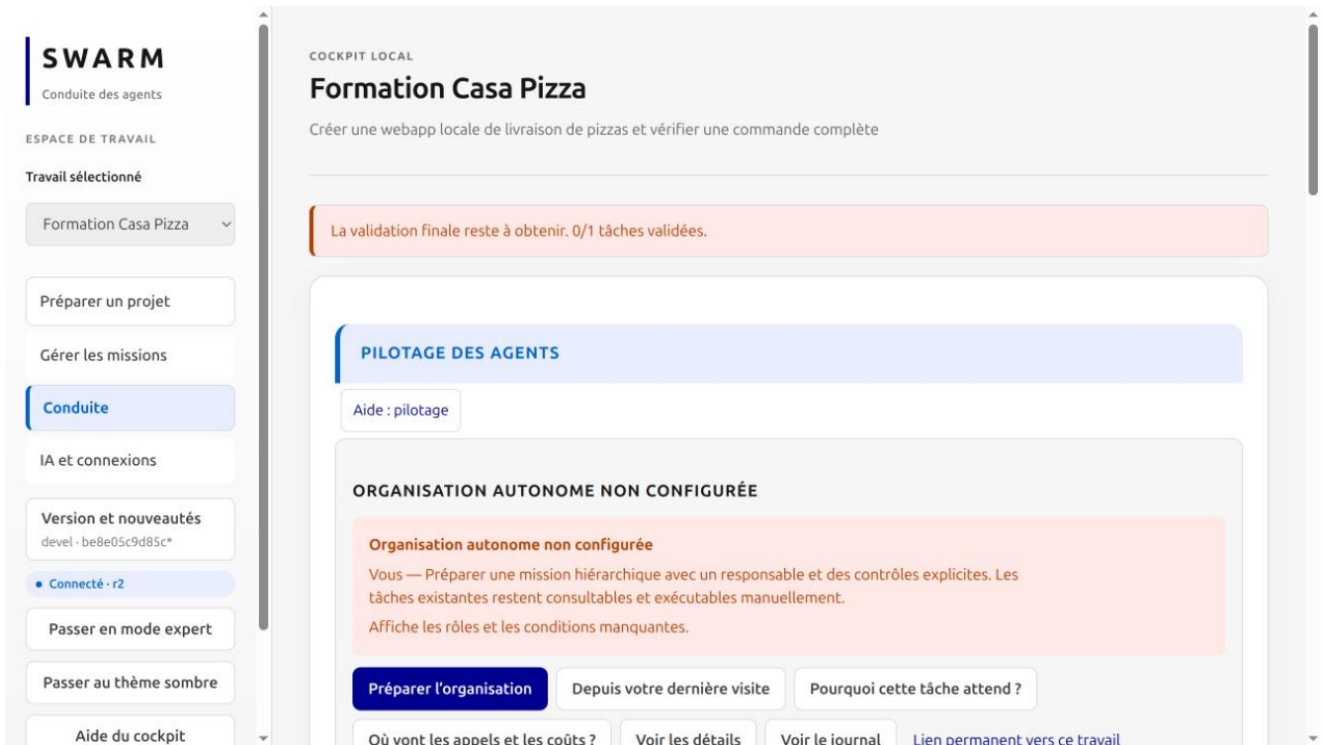
The requirement is entered in Prepare a project. Saving retains the text; it does not start an executor.

1. Describe users, the problem, observable outcomes and exclusions.
2. Review the brief and proposed tasks before adoption. AI text remains a proposal.
3. Then verify in Overview that tasks exist with criteria and deliverables.
4. Configure the executor and confirm its launch separately.

The kit provides a reproducible shortcut: `atelier_swarm.py` creates work and one task from the supplied requirement. The workshop does not claim to have executed every assisted preparation step or a hierarchical mission.

10 Read the guided mission

Select Formation Casa Pizza in Travail sélectionné, meaning Selected work. Open Conduite, the control view, and read the objective before launching anything.



The cockpit shows zero of one task validated. Organisation autonome non configurée refers to hierarchical automation.

1. Check that Formation Casa Pizza is selected.
2. Scroll to Construire la webapp et ses tests, meaning Build the webapp and its tests.
3. Open the task and check its deliverables: application, tests, README, and report.

Why the organization warning appears

This mission is operated manually with one executor. The banner says that a responsible agent and the controls for an autonomous organization have not been configured. Existing tasks remain executable manually, as the message explains. This journey is not a demonstration of autonomous hierarchical operation.

Executor or API connection

A chat API connection can help draft text. Producing project files requires an executor with tools. In this workshop, Codex is that executor program, configured by the kit.

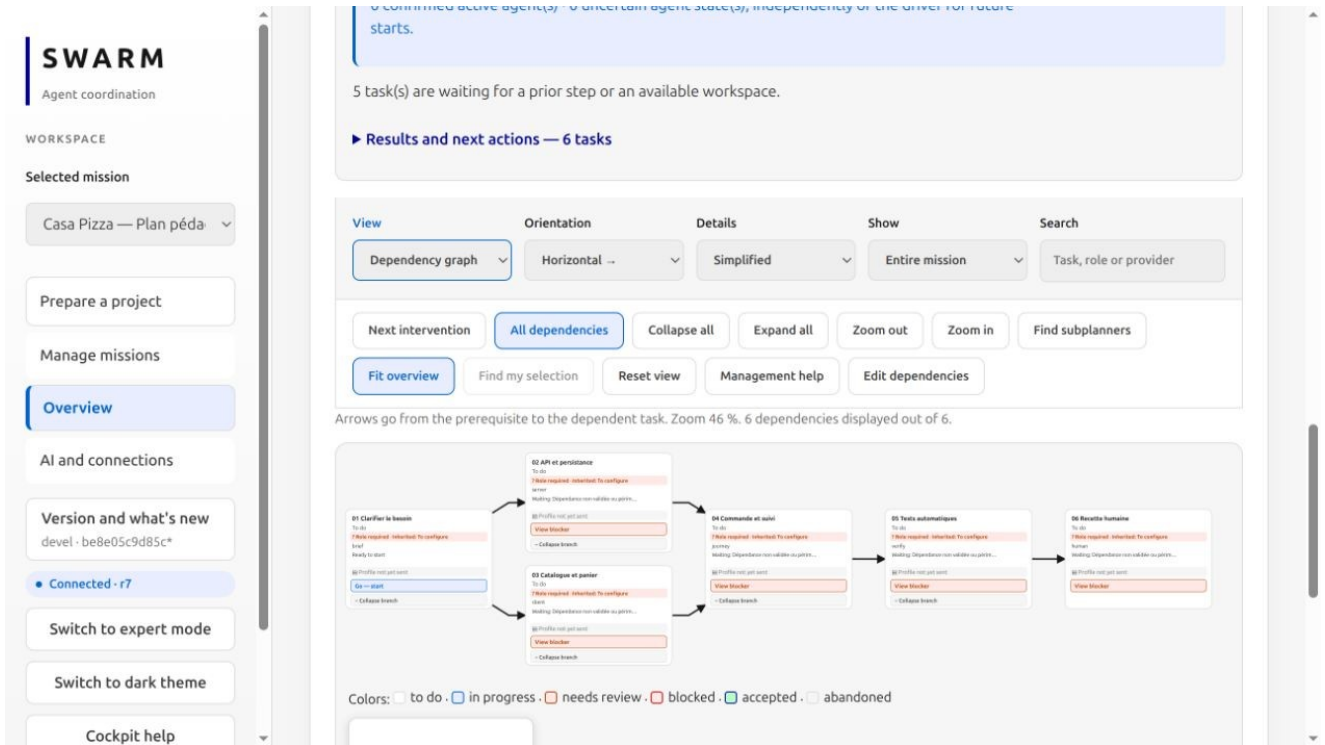
Expected result. You can identify the single task and the program that will create its files.

11 Read the task graph

Select Casa Pizza — Plan pédagogique in the mission selector. This is the optional demonstration plan in the kit, separate from the original one-task mission.

To create this optional plan, run the command once from the kit after creating your project. It adds demonstration work without changing the original mission or starting an agent.

```
python3 creer_plan_pedagogique.py \
--swarm /path/to/swarm/bin/swarm \
--project "$HOME/projects/pizza-workshop"
```



Actual cockpit capture with six unexecuted tasks. The six arrows represent their prerequisites.

1. Find 01 Clarifier le besoin. It has no prerequisite.
2. Follow the arrows to 02 API et persistance and 03 Catalogue et panier.
3. Find the merge at 04 Commande et suivi: both prerequisites are needed.
4. Read the remaining chain: 05 Tests automatiques, then 06 Recette humaine.

An arrow goes from a prerequisite to its dependent task. Two branches can become eligible after the same prerequisite; concurrent execution also depends on slots, budgets, roles and available workspaces.

Expected result. You can explain why Order and tracking waits for both branches. No task in this plan has been launched.

12 Read a task detail

Click 04 Commande et suivi. The detail brings together state, criteria, dependencies, permitted actions and reports.

The screenshot shows the SWARM interface with a sidebar on the left and a main workspace. The sidebar includes sections for 'Agent coordination', 'Workspace', 'Selected mission' (Casa Pizza — Plan péda), and various action buttons like 'Prepare a project', 'Manage missions', 'Overview' (highlighted), 'AI and connections', 'Version and what's new', 'Connected - r7', 'Switch to expert mode', 'Switch to dark theme', and 'Cockpit help'. The main workspace displays a 'Dependency graph' view with a horizontal orientation and a simplified detail view. A task detail panel for '04 Commande et suivi' is open on the right, showing missing prerequisites (server, client), a list of dependencies, and a section for 'What should I do now?' with authorized actions and task reports.

The detail describes missing prerequisites. Scroll the panel to read Dependencies and impact and authorized actions.

Signal	What it means
To do	The task remains to be done; it may still be ineligible to start.
Ready to start	Observed conditions allow a launch to be prepared.
Waiting	A prerequisite, evidence or workspace is missing. Read the reason.
Needs review	A result needs evaluation; it is not yet accepted.
Accepted	The result has acceptance; also read process history.

Always distinguish task state, process state and validation state. A finished or stopped process is not proof of a correct delivery. An interrupted attempt can remain in the history of a result that is later corrected and accepted.

13 Explore and edit the graph

View controls make the graph easier to read. Editing dependencies changes the work organization and requires an additional step.

The screenshot displays the SWARM interface. On the left is a sidebar with the 'SWARM' logo, 'Agent coordination', and a 'WORKSPACE' section containing 'Selected mission' (Casa Pizza — Plan péda), 'Prepare a project', 'Manage missions', 'Overview' (highlighted), 'AI and connections', 'Version and what's new', 'Connected - r7', 'Switch to expert mode', 'Switch to dark theme', and 'Cockpit help'. The main area shows a 'Who acts' section with 'You', an 'Expected result' section with buttons for 'Review prerequisite', 'Configure validation', 'Explain with AI', and 'Why is this task waiting?'. Below this is a control bar with tabs for 'View' (Dependency graph), 'Orientation' (Horizontal), 'Details' (Simplified), 'Show' (Entire mission), and 'Search' (Task, role or provider). A row of buttons includes 'Next intervention', 'All dependencies' (active), 'Collapse all', 'Expand all', 'Zoom out', 'Zoom in', and 'Find subplanners'. Another row has 'Fit overview', 'Find my selection', 'Reset view', 'Management help', and 'Edit dependencies'. A text line states: 'Arrows go from the prerequisite to the dependent task. Zoom 46 %. 6 dependencies displayed out of 6.' The bottom section is the 'Edit dependencies' modal, which includes a 'Leave editing' button, instructions to 'Choose an operation, then the prerequisite and dependent task in the graph. Enter and Space provide the same path as the mouse.', and three dropdown menus for 'Operation' (Add a dependency), 'Prerequisite' (Choose a task), and 'Dependent task' (Choose a task). Below these are buttons for 'Add to draft', 'Cancel', 'Redo', 'Preview', and 'Apply explicitly'. A status message at the bottom reads: '0 change(s) in the draft. The mission is unchanged. 1. No prepared change.'

The editor provides a prerequisite, a dependent task and a draft. This capture retains zero changes; nothing was applied.

Explore without changing tasks

Fit overview reframes the graph. Zoom helps read a card. Orientation changes its reading direction. Filters show part of the work and search helps locate a task. These display actions neither accept nor launch tasks.

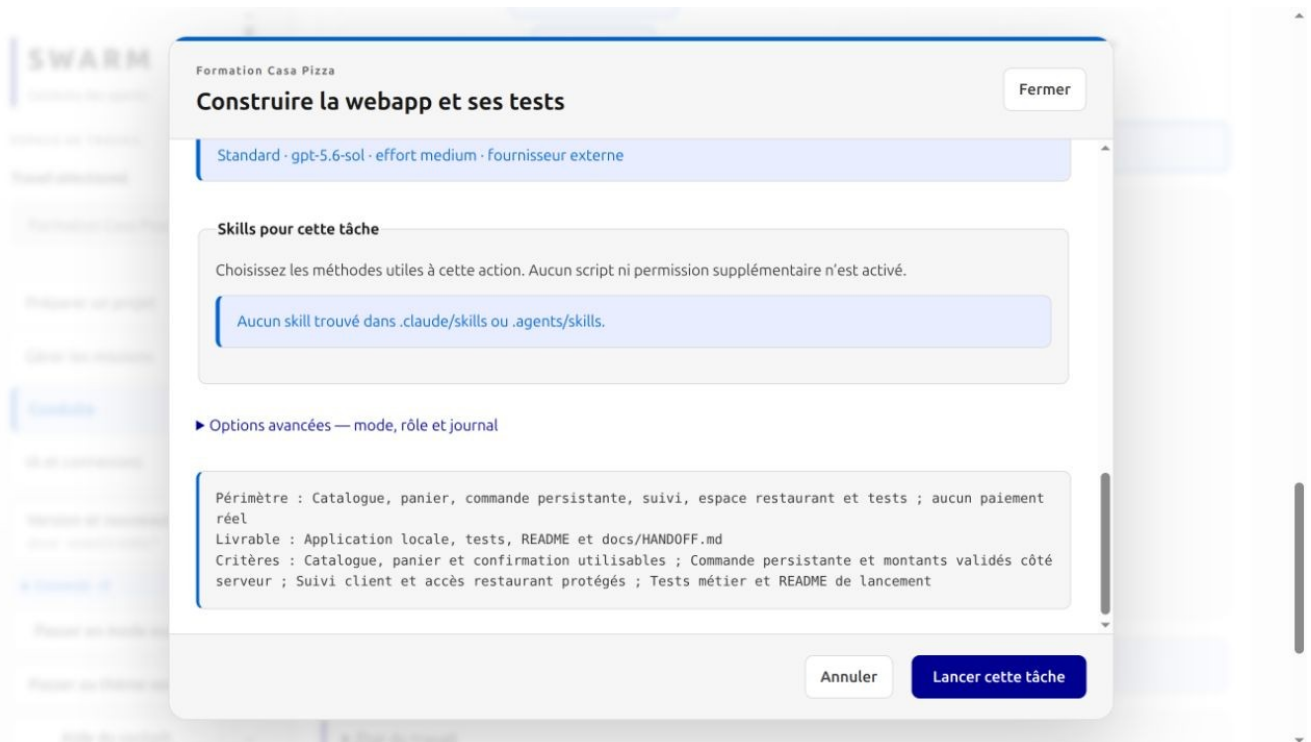
Understand a change

Edit dependencies prepares a draft. The operation specifies Add or Remove, followed by the prerequisite and dependent task. Preview lets you examine the effect before Apply explicitly. Undo and Redo affect the draft; they do not reverse work already executed.

For this workshop, keep the six planned dependencies. Do not remove a prerequisite simply to hide a wait. A hidden branch or a different color is not proof that the corresponding work is complete.

14 Launch a task with clear limits

Open the task launch dialog. Read the instruction, project folder, and acceptance criteria before confirming.



The bottom of the dialog lists scope, deliverables, and criteria. **Lancer cette tâche** means **Launch this task and starts the executor**.

1. Choose Codex from the available executors. Check the pizza-workshop folder.
2. Keep the task criteria. No additional skill is required by the kit.
3. Click **Lancer cette tâche**, then open **Voir la session**, meaning **View the session**.

Instruction supplied by the kit

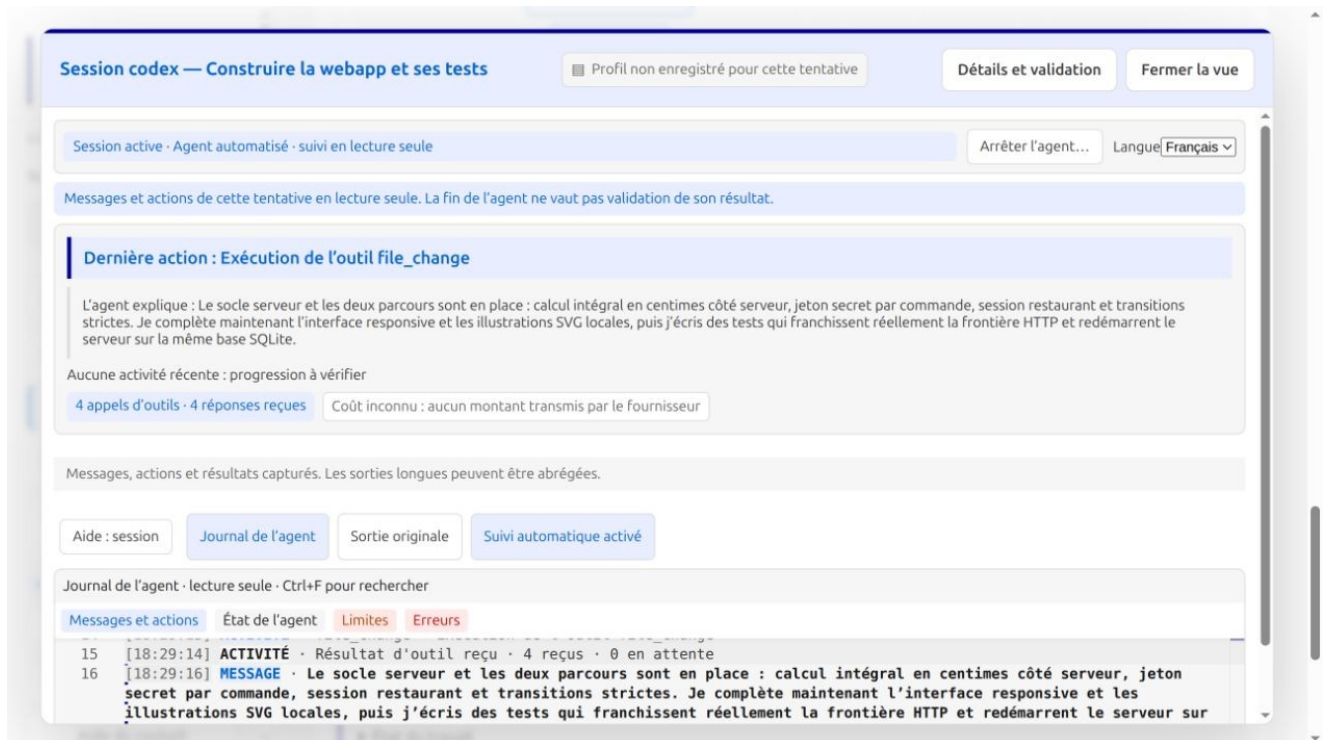
Implement docs/BESOIN.md and follow AGENTS.md. Deliver code, tests, README, and docs/HANDOFF.md. Use the Python standard library, SQLite, and local resources. No other agent and no publication. Do not start a long-running server. Check totals, errors, access controls, and persistence. Do not change .swarm.

The kit stores this instruction in French. The screenshot shows the model requested for this workshop. It depends on the provider configuration and is not proof of the model actually served.

Expected result. A new attempt appears in the session. You can inspect its messages and actions.

15 Monitor an execution

During generation, use the session to distinguish messages, tool calls, and errors. Generated code still needs verification.



The session shows a file_change action and four tool responses. The agent's message describes work in progress.

1. Read Dernière action, meaning Latest action, to see what the agent just did.
2. Open Journal de l'agent, the agent log, then Messages et actions.
3. If progress appears stalled, inspect État de l'agent, Limites, and Erreurs: agent state, limits, and errors.

Aucune activité récente means No recent activity: investigate progress. Coût inconnu means Cost unknown: the provider did not report an amount. It does not mean that the call was free.

Session completion and delivery

Wait for the delivery report. Check the files and the tests actually run. In the observed workshop, the agent produced the code and then stopped after repeated structured handoff errors. The trainer subsequently inspected, corrected, and tested the result on the host computer.

Expected result. You can explain the observed actions without treating an agent's statement as proof that the application works.

16 Resolve a startup failure

The first attempt failed before producing code. Its original output said Not inside a trusted directory.

1. Open the failed session and select Sortie originale.
2. Find the first explicit cause. Here, the project folder was not a Git repository.
3. Check the project folder. The kit script initializes Git to avoid this problem.
4. After correcting the cause, resume the same task to preserve the attempt history.

Check inside the project folder

```
git status
```

If you prepared a fresh folder manually and Git says it is not a repository, the trainer can initialize that folder with `git init` and commit the requirement files. Do not initialize an unrelated project folder by guesswork.

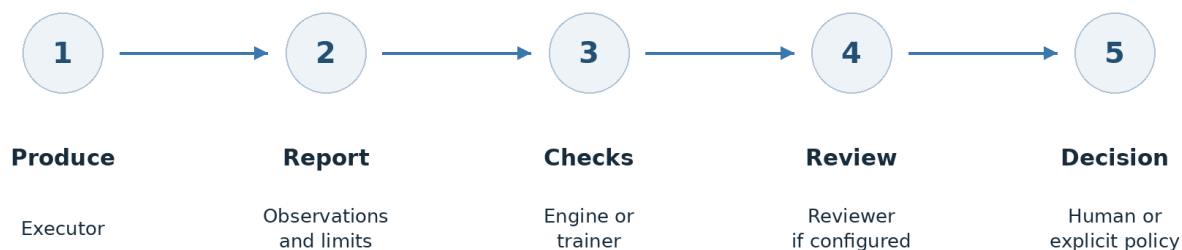
A limit or repeated identical errors calls for diagnosis. Do not increase budgets to hide a blockage. If the sandbox prohibits sockets, run the HTTP tests on an authorized local host and record that distinction.

Expected result. The cause is identified, the fix is limited to the project, and the failure remains visible in the history.

17 Distinguish verification stages

“Self verification” can refer to different operations. Identify the actor and evidence before drawing a conclusion.

FROM PRODUCTION TO A DECISION



Each stage provides different information about the result

Explanatory diagram • Checks alone do not establish qualitative quality

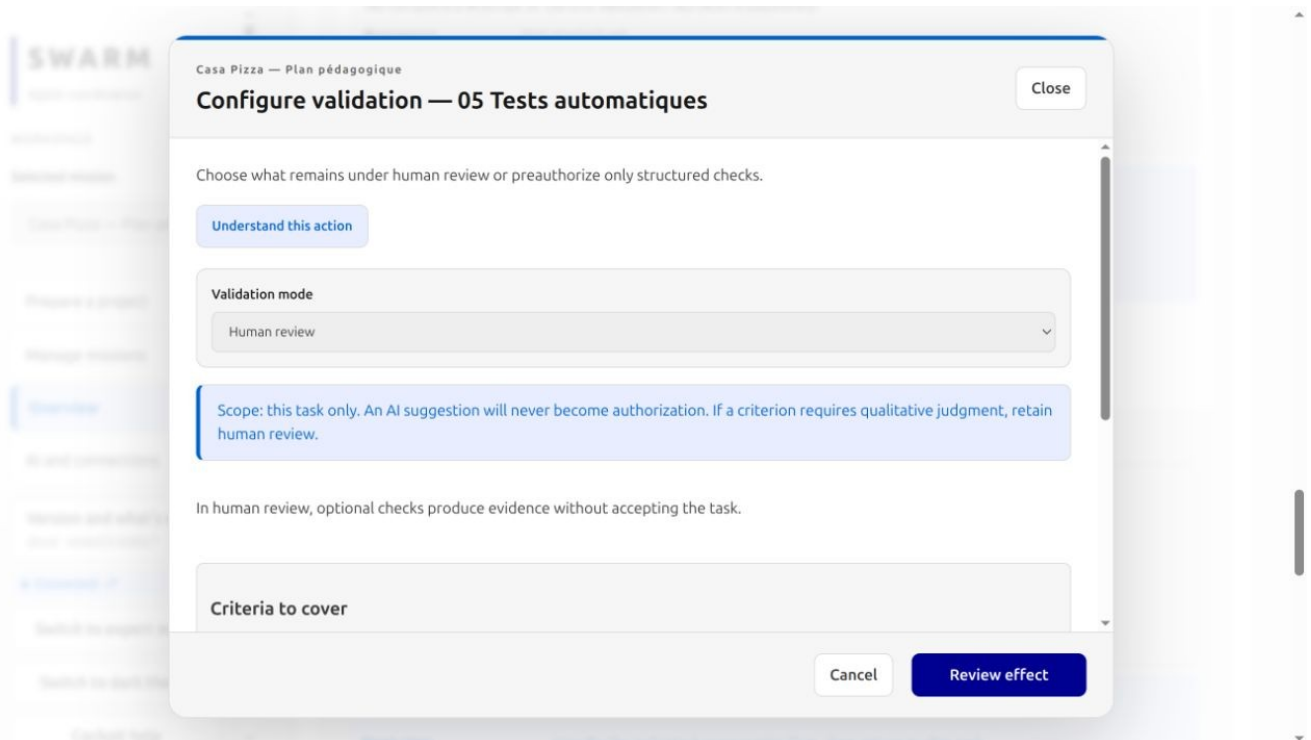
Producer observations, checks and the final decision serve different purposes.

Stage	Expected evidence	Limit
Producer tests	Actual command, output and result	Not independent review
Engine checks	Receipt tied to candidate and declared files	Covers declared testable criteria
Independent review	Verdict in a separate session	A reviewer judgment to examine
Human acceptance test	Observed journey and report	Only the actually tested scope

In the original mission, the producer ran four application tests. The trainer added seven HTTP tests, ran all eleven and replayed the browser journey. These checks were not launched by an automatic Swarm policy.

18 Configure human review with checks

The following example shows a configuration preview for 05 Tests automatiques. It authorizes no continuous mission and starts no agent.



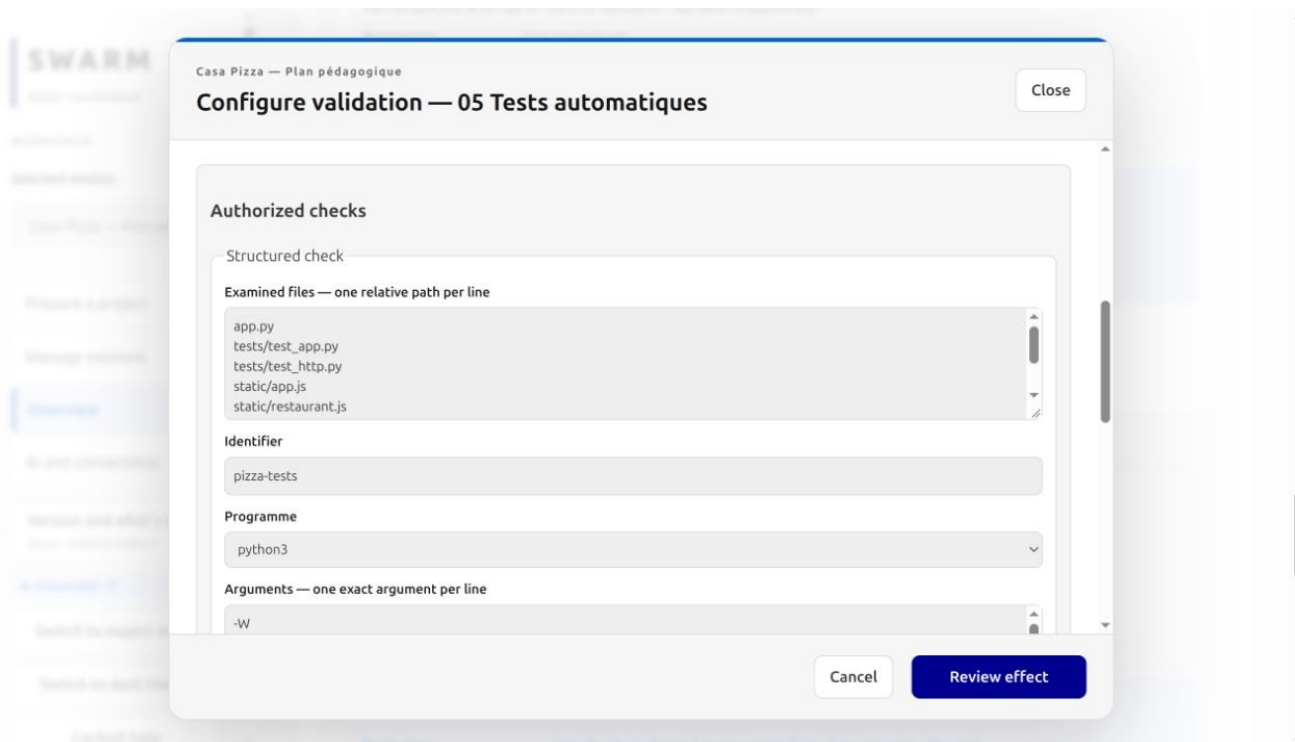
Human mode retains the acceptance decision. Optional checks can provide evidence.

1. In Overview, click View details to open Results and next actions.
2. For 05 Tests automatiques, choose Configure validation. The gate button in the task detail is a different operation.
3. Keep Human review. Read the criteria before adding a check.
4. Add only a command actually available in the environment running Swarm.

If a criterion requires qualitative judgment, retain human mode for the entire task. Commands can support review without replacing the person. An AI suggestion alone is never authorization.

19 Define the test check

This example uses the Casa Pizza reference suite. For your own candidate, first verify its files, test count and applicable command.



The program and arguments are separate. No implicit shell turns these lines into a script.

Identifier: pizza-tests. Program: python3. Directory: . (project root). Timeout: 60 seconds. Arguments, one per line:

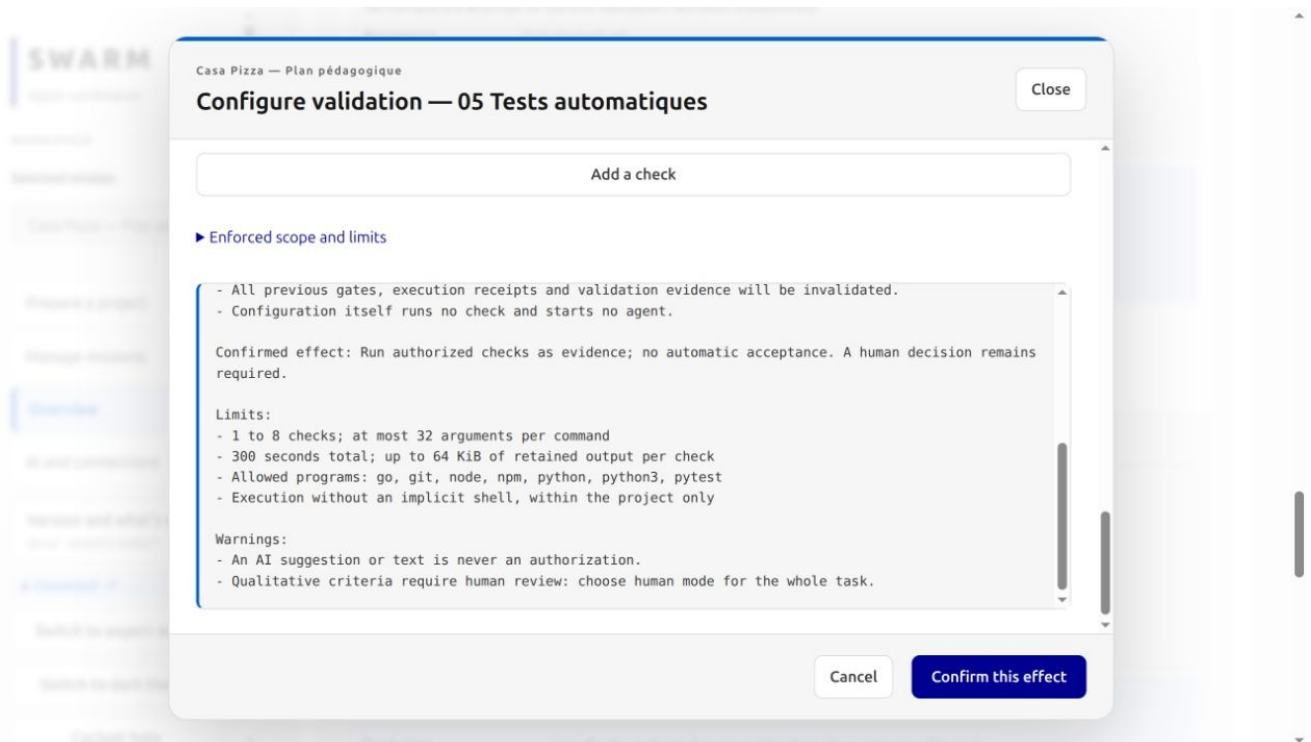
```
-W
error::ResourceWarning
-m
unittest
-v
```

Select only the criterion “The candidate’s eleven tests pass”. Justify coverage using the behaviors those tests exercise. This check alone does not establish usability, complete accessibility or production readiness.

Examined files: app.py, tests/test_app.py, tests/test_http.py and the HTML, JS and CSS used. Paths are relative to the project root even if the command directory differs. Also declare every other file on which your verdict depends.

20 Review consequences before confirmation

Click Review effect. The preview lets you check the exact scope before recording the policy.



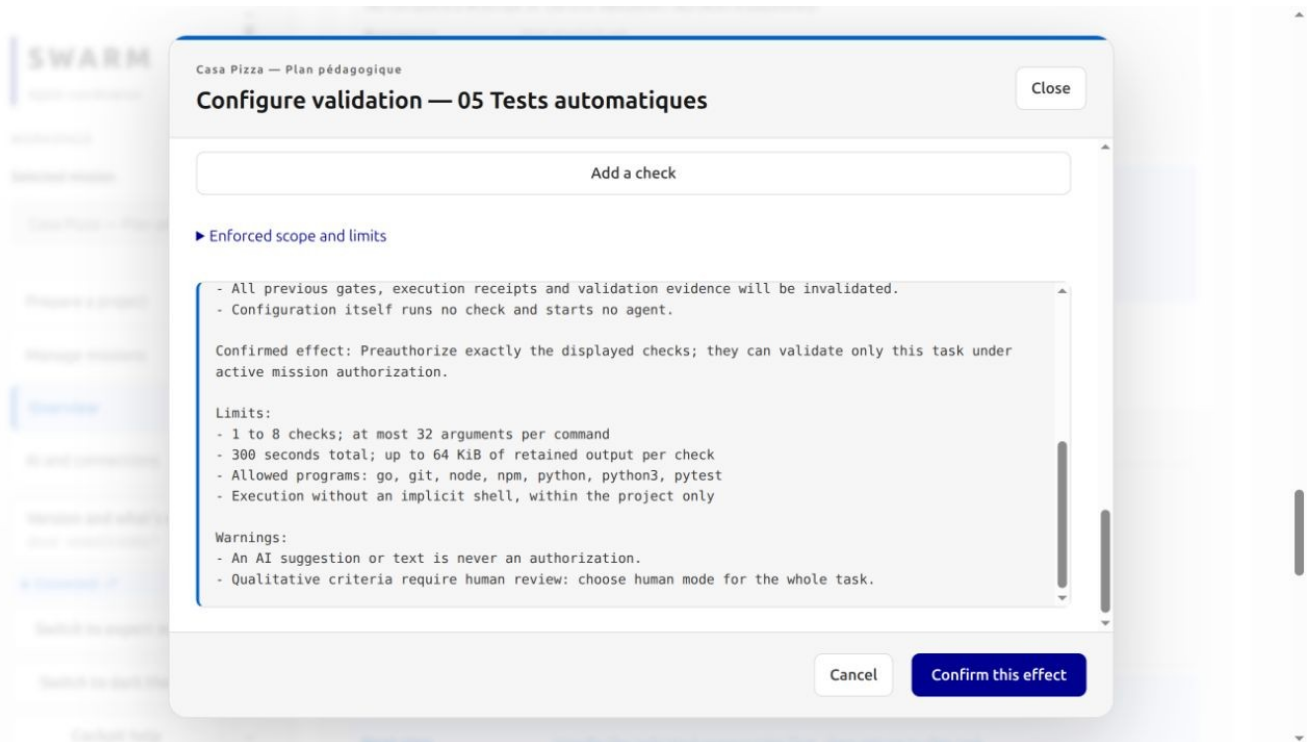
Human-mode preview. Configuration starts neither a check nor an agent and does not provide automatic acceptance.

1. Review the task identifier, command, files, criteria, directory and timeout.
2. Verify that a human decision remains required.
3. Read the invalidation of earlier gates and evidence. A changed policy needs a new evaluation applicable to the candidate.
4. Confirm only if this is the intended effect. For the guide demonstration, Cancel was chosen.

Sharing output with the reviewer is optional and limited to 8 KiB. Output may contain sensitive information; the example leaves this option unchecked. An incomplete file list limits what the evidence establishes.

21 Understand automatic validation

Automatic mode applies to objective criteria fully covered by explicitly preauthorized checks. The preview below was not confirmed.



Actual automatic-mode preview. Its scope remains this task, under active mission authorization.

Recording a policy runs no check. The engine requires, among other conditions, an authorized continuous mission that is not paused, the current completed production attempt and a submitted result. Criteria must be covered; programs and timeouts are bounded.

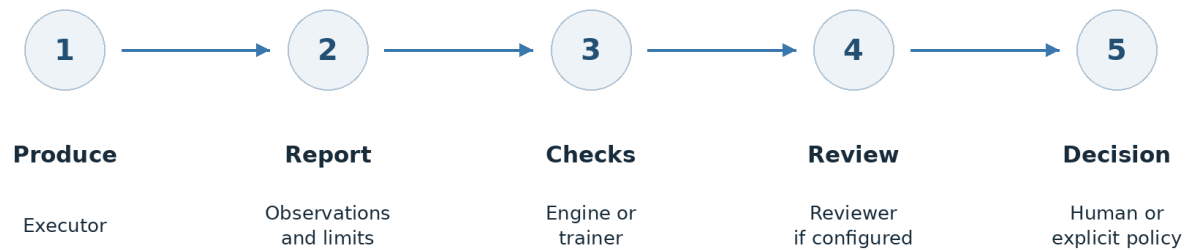
A failed check prevents acceptance. A successful check may still leave an independent review required. In human mode, it still requires a human decision. Acceptance depends on all applicable guards, not simply an "OK" message.

The engine allows 1 to 8 checks, at most 32 arguments per command, 300 seconds total and up to 64 KiB of retained output per check. This configuration illustrates the mechanism; the original mission remains a guided workshop.

22 Check that evidence is current

A successful check concerns a specific candidate. It does not automatically cover a file changed afterwards.

FROM PRODUCTION TO A DECISION



Each stage provides different information about the result

Explanatory diagram • Checks alone do not establish qualitative quality

Acceptance must concern the candidate actually checked and reviewed.

1. Identify the task, attempt and relevant report.
2. Inspect the command, exit code, errors and output limits.
3. Check declared files and their hashes. Without declared files, a receipt bound only to the report does not establish that all code remained unchanged.
4. After changing code or policy, repeat the necessary checks and applicable review.

A current receipt can be reused without rerunning a command on every driver pass. History remains when evidence becomes stale. A check that could not execute does not establish a business defect in the code: diagnose its environment first.

Example

All eleven tests pass. You then edit `app.py`. The previous result is insufficient for the new candidate. Rerun the suite and affected journey, then update the report before requesting acceptance.

23 Keep a human in the loop

A person intervenes when a decision changes scope, execution or acceptance. They need a concrete result and readable evidence.

Moment	Inspect	Human decision
Before the plan	Requirement, exclusions, observable criteria	Adopt or correct
Before launch	Tools, folder, instruction, limits, requested model	Authorize the intended launch
Before a policy	Exact commands, files, scope and invalidation	Confirm or cancel
After delivery	Actual tests, journey, report and limitations	Accept or request correction
After a block	Cause, relevant attempt, recovery effect	Resume with a targeted action

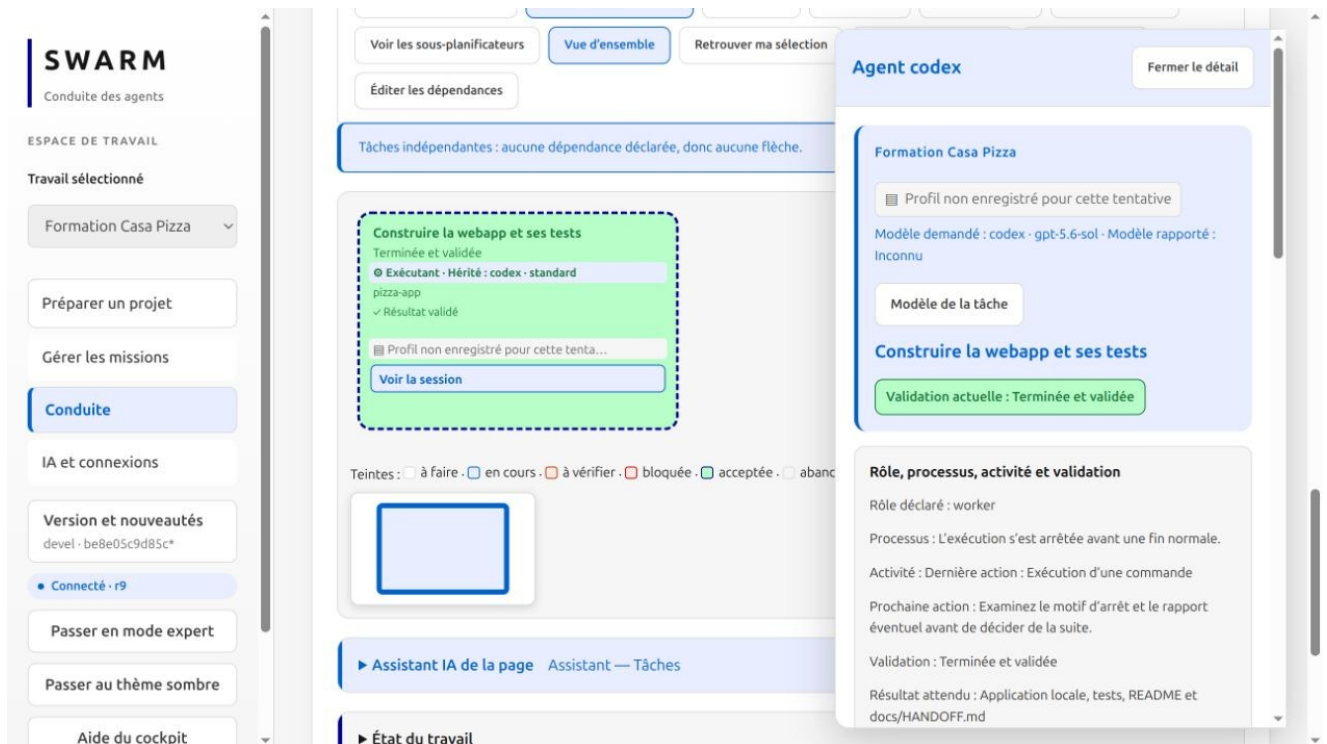
Acceptance questions for Casa Pizza

Is the total understandable? Do errors explain how to fix the order? Can buttons be used with a keyboard? Does tracking show the same order as the restaurant? Is the journey readable at 390 pixels? Are excluded functions explicitly stated?

Record the observation and limitation. "It works" is too vague. Prefer "two Margherita and one Reine, total 34.40 euros, same restaurant order number, Delivered visible after refresh".

24 Understand independent review

Independent review provides a separate examination of the candidate. It requires actual configuration and cannot be inferred from a task name.



The original workshop retains trainer acceptance and an interrupted process. This capture establishes no independent review.

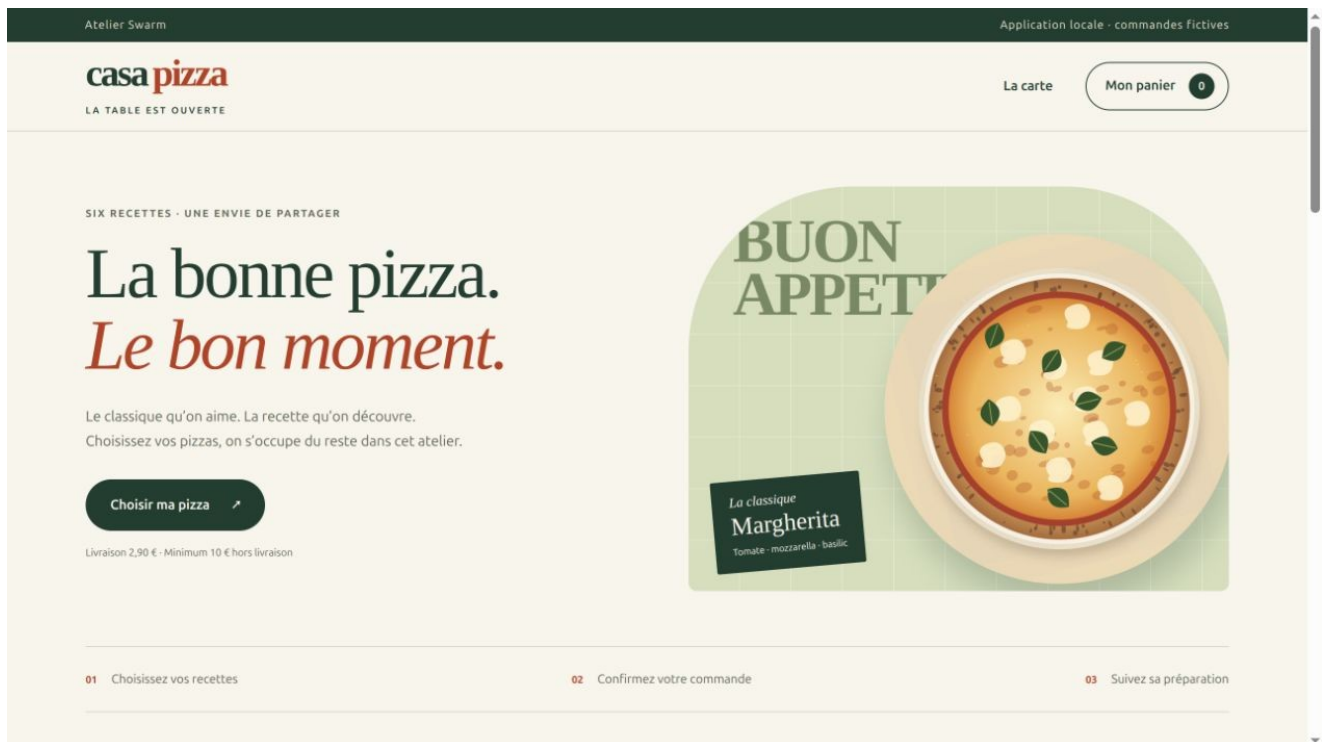
The reviewer needs the report and relevant evidence in a separate session with scope and limits. The verdict must concern the examined candidate. The reviewer must not silently produce fixes and then judge its own work.

A favorable verdict does not replace a failed mandatory check. A rejection needs a targeted correction and fresh evaluation. Stale evidence needs review applicable to the current candidate.

For this guide, the trainer performed the original verification within the delivery session. It is explicitly described that way. Diagrams explain possible responsibilities; they do not claim to demonstrate an executed autonomous hierarchy or independent review.

25 Give the design a direction

Functional requirements do not establish visual quality. The first brief specified colors and cards, but the first composition remained too generic for a public presentation.



The reference homepage was refined in the trainer session after the initial workshop. This illustrated design uses local assets only.

A more specific instruction

Create an illustrated trattoria identity: cream paper, dark green ink, terracotta accents, serif headings and restrained utility text. Compose an original homepage with detailed local artwork and a clear menu button. Distinguish all six recipes. Carry this identity into the basket and tracking. Make prices and errors clear. Check mobile layout and keyboard focus. Do not invent customer ratings, sourcing claims or delivery times.

Reference images are SVG illustrations, not photographs. Local assets keep the workshop free of external downloads. Another visual direction requires its own brief and acceptance review.

26 Review visual quality

Visual quality requires a human decision on the actual rendering. Passing business tests does not automatically make a page attractive or distinctive.

casa pizza
LA TABLE EST OUVERTE

La carte Mon suivi **Mon panier 3**

Votre commande.

Une dernière vérification avant de confirmer.

Votre sélection

01 / PANIER

Margherita
9,50 € pièce

Quantité: 2

Reine
12,50 € pièce

Quantité: 1

Sous-total: 31,50 €

Livraison: 2,90 €

Total: 34,40 €

Minimum de commande : 10,00 € hors livraison.

Où vous livrer ?

02 / COORDONNÉES

Pour cet atelier, utilisez uniquement des coordonnées fictives.

Nom

Adresse fictive

Téléphone fictif

Confirmer la commande

Paiement à la livraison simulé - aucun paiement réel

The reference basket separates selection, amounts and contact details. Review examines readability and the journey beyond whether a click works.

Review point	Reviewer action
Hierarchy	Find the main action and price without searching
Consistency	Compare homepage, menu, basket and tracking
Mobile and keyboard	Test at 390 px and follow visible focus
Product truth	Check messages, amounts and claims

The current refinement and its fresh verification are recorded in `projet-pizza/docs/REFONTE.md`. Swarm acceptance screenshots show the historical version. They do not establish acceptance of this revision. The trainer repeated the browser journey and eleven tests; no independent review of the refinement is claimed.

27 Check the menu and basket

Once the code is available, open a second terminal in pizza-workshop. Run `python3 app.py` and open `http://127.0.0.1:18841/`. Stop the reference server first if it occupies that port.

casa pizza
LA TABLE EST OUVERTE

La carte Mon suivi **Mon panier 3**

Votre commande.

Une dernière vérification avant de confirmer.

Votre sélection 01 / PANIER

	Margherita 9,50 € pièce	Quantité 2
	Reine 12,50 € pièce	Quantité 1
Sous-total		31,50 €
Livraison		2,90 €
Total		34,40 €

Minimum de commande : 10,00 € hors livraison.

Où vous livrer ? 02 / COORDONNÉES

Pour cet atelier, utilisez uniquement des coordonnées fictives.

Nom

Adresse fictive

Téléphone fictif

Confirmer la commande

Paiement à la livraison simulé - aucun paiement réel

Two Margherita pizzas and one Reine total 31.50 euros, plus 2.90 euros delivery, for a total of 34.40 euros.

1. Check all six recipes. Select *Végétariennes uniquement*, meaning Vegetarian only: only Margherita, Quatre fromages, and Végétarienne should remain.
2. Clear the filter. Add a Margherita and a Reine.
3. Open *Mon panier*, then change the Margherita quantity to 2. You can use the arrow keys in the quantity field.
4. Enter Camille Exemple, 12 rue de la Démonstration, and 0600000000. These are fictional details.

Expected result. The total is 34.40 euros. Quantity controls and fields are usable with a keyboard.

The server validates the values. The browser provides a convenient display calculation, but it must not be able to impose a forged price.

28 Test a useful error

Before confirming a valid basket, check that the product explains a refusal. This exercises the minimum-order rule.

The screenshot shows the Casa Pizza checkout interface. On the left, a Margherita pizza is listed with a price of 9.50 € and a quantity of 1. The subtotal is 9.50 €, and the delivery fee is 2.90 €, resulting in a total of 12.40 €. A note indicates a minimum order of 10.00 € excluding delivery. On the right, the checkout form is filled with fictional details: Name (Camille Exemple), Address (12 rue de la Démonstration), and Phone (0600000000). A red error message states: "Le minimum hors livraison est de 10,00 €." The "Confirmer la commande" button is disabled. The footer shows "casa pizza" and "Démonstration locale - sans paiement réel".

One Margherita costs 9.50 euros before delivery. The 12.40 euro total therefore does not meet the minimum for products.

1. In a fresh basket, add only one Margherita.
2. Enter fictional details, then click Confirmer la commande, meaning Confirm the order.
3. Check the error Le minimum hors livraison est de 10,00 euros: the minimum excluding delivery is 10 euros.
4. Add another pizza or increase the quantity. The error should disappear when the basket is corrected.

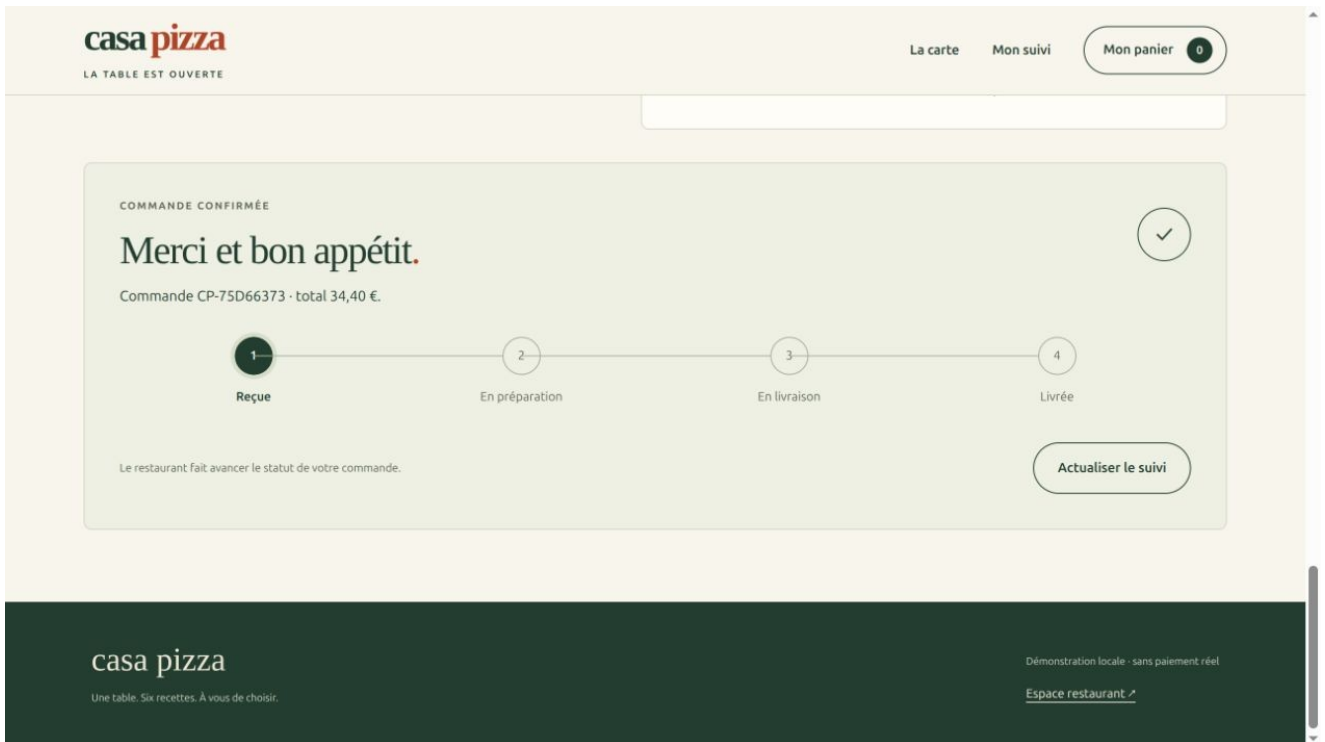
Expected result. The server refuses the insufficient order. The rule applies to products, not delivery charges.

Two more checks

Try an empty name and an invalid quantity. The order must remain blocked with a clear error. Correct the inputs before continuing the valid journey.

29 Confirm and track an order

Rebuild the basket with two Margherita pizzas and one Reine. A confirmation should display an order number, total, and initial status.



Reference order CP-75D66373 was received for 34.40 euros. Your order number will be different.

1. Click Confirmer la commande once.
2. Record the displayed order number and check the 34.40 euro total.
3. Keep this tab open to retain private tracking.
4. Use Actualiser le suivi, meaning Refresh tracking, after a restaurant status change.

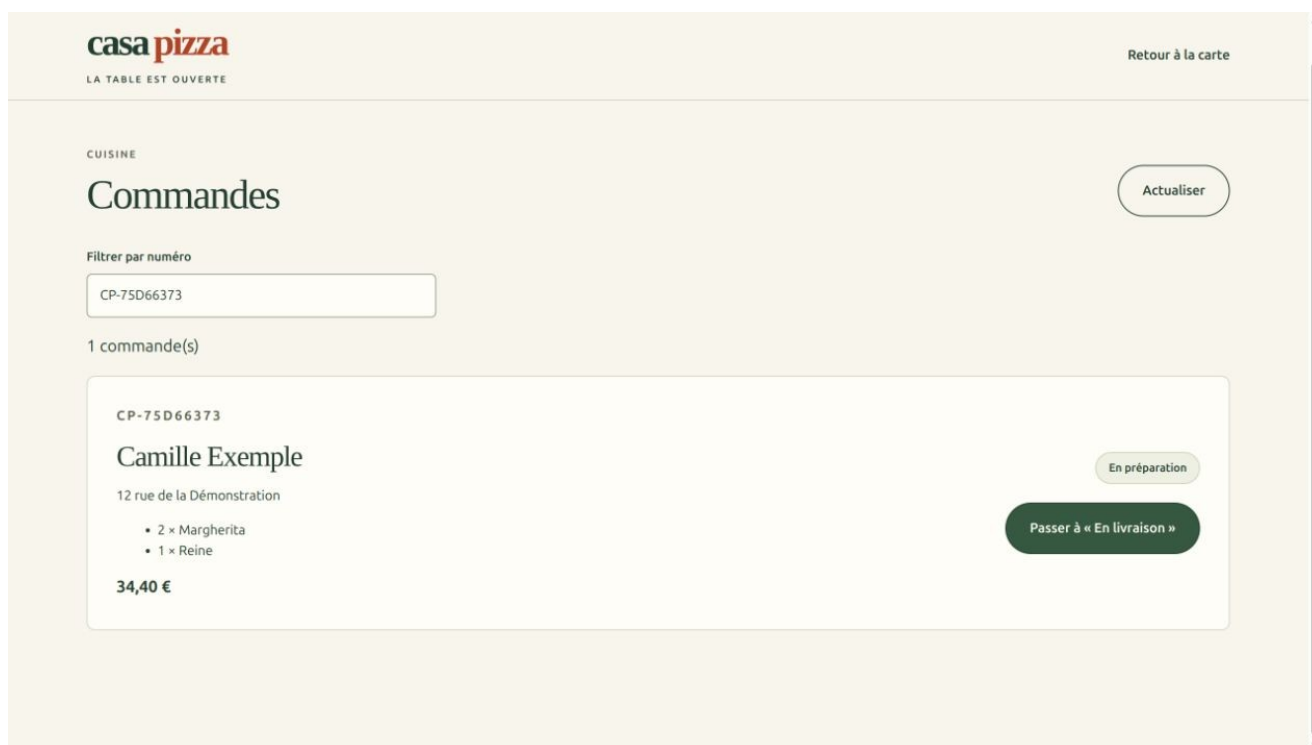
Expected result. The initial status is Reçue, meaning Received. A visible new number and correct total confirm creation of this order.

The browser retains a private tracking token in its session. The number alone does not let another browser read the order. Do not copy tokens or real contact details into a training report.

The server stores orders in SQLite. Restarting it with the same database preserves orders, but invalidates restaurant sessions held in memory.

30 Update the restaurant status

Open <http://127.0.0.1:18841/restaurant> in a second tab. You are now playing the role of the kitchen.



The same order contains two Margherita pizzas and one Reine. It is being prepared; the button offers the next delivery stage.

1. Try an incorrect password once: access must be refused.
2. Sign in with atelier if you use the local demo configuration.
3. Find your order number, optionally using Filtrer par numéro to filter by number. Move it from Reçue to En préparation: Received to Preparing.
4. Move it next to En livraison, then Livrée: Out for delivery, then Delivered.

Expected result. The restaurant displays products, total, and status. Each button offers only the next stage.

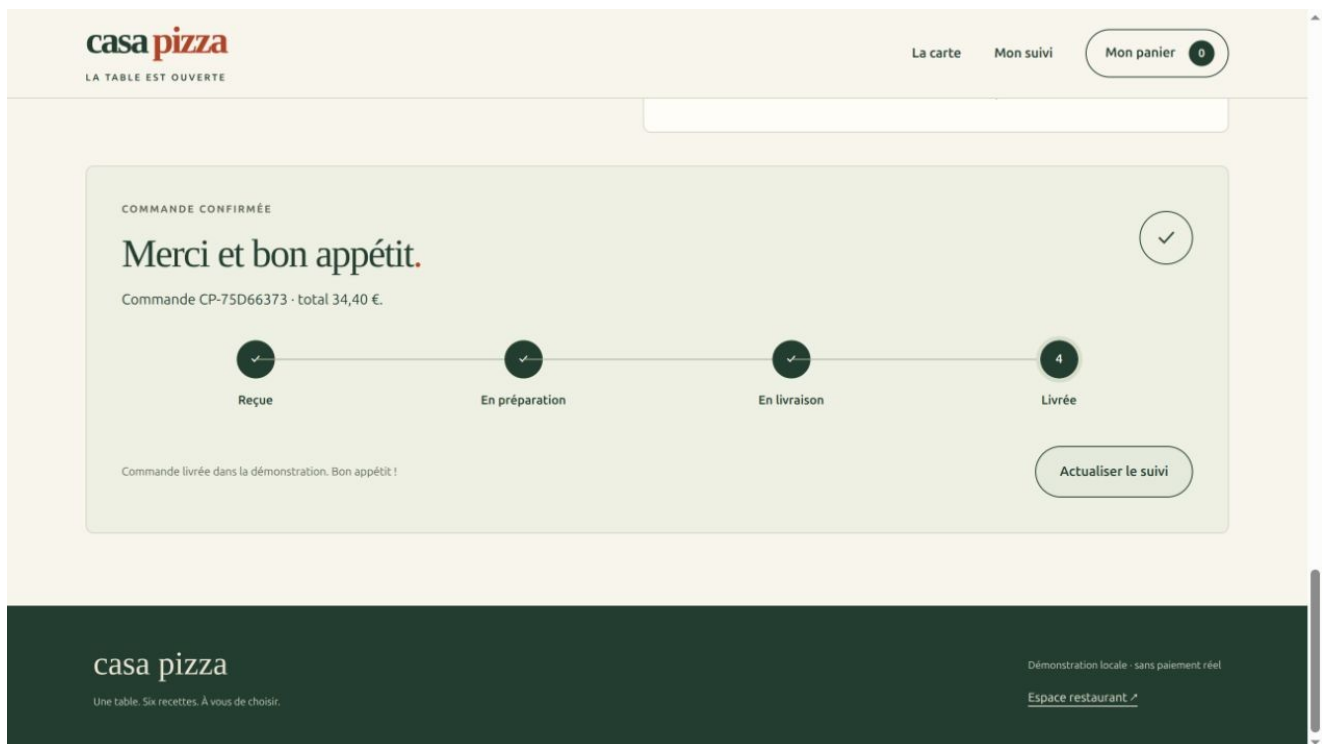
Allowed stages are Received → Preparing → Out for delivery → Delivered. The server rejects inconsistent transitions. HTTP tests also check unauthenticated access and incorrect passwords.

Keep this application on the local loopback interface. The workshop's shared password and in-memory sessions are not a production setup.

Progression is manual: a person uses buttons in the restaurant view. Statuses do not advance automatically with time. A received order therefore stays Received until that action is performed.

31 Check final tracking and mobile use

Return to the client tab and click Actualiser le suivi. The status must match the restaurant status.



All four stages are green after the reference order reaches **Livrée**, meaning **Delivered**.

1. Compare the client and restaurant order numbers.
2. Check that Delivered is reached after the three transitions.
3. Repeat an order in a browser viewport 390 pixels wide, or on a phone using a local environment prepared by the trainer.

Expected result. Tracking reflects the server. At 390 pixels, cards, fields, and buttons remain usable without horizontal scrolling.

The reference was tested in a browser set to 390 pixels. A separate mobile order containing one Reine was received for 15.40 euros. Its number and basket therefore differ from the desktop journey.

32 Inspect a mobile confirmation

The mobile check used an actual 390-pixel browser viewport. This screenshot shows a different order from the desktop scenario.



One Reine plus delivery costs 15.40 euros. The tracking stages wrap onto two rows.

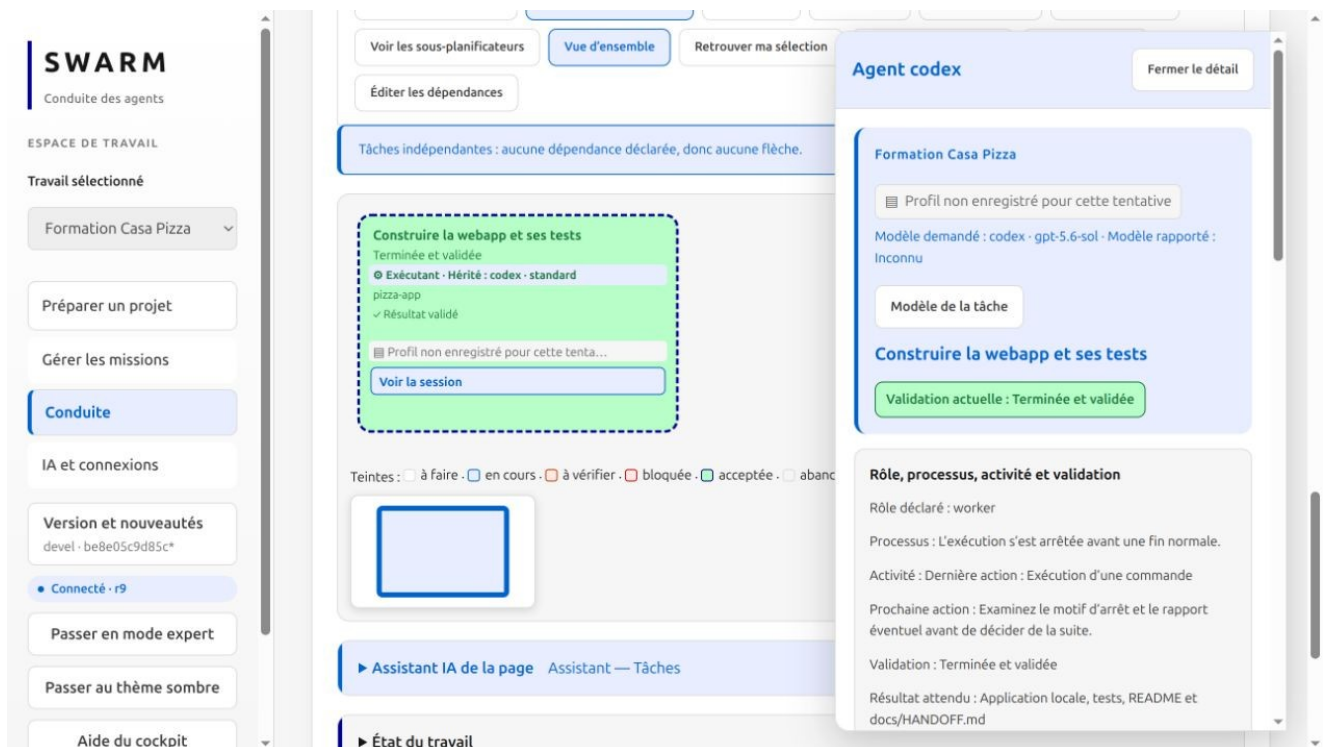
1. Add one Reine to a fresh basket.
2. Enter fictional details and confirm.
3. Check the number, total, and Refresh tracking button without horizontal scrolling.

Expected result. The confirmation remains readable and usable on a narrow screen.

33 Verify before accepting

The delivered reference passes eleven tests: four application tests and seven HTTP tests using a real local server, including a restart on the same SQLite database.

```
python3 -W error::ResourceWarning -m unittest -v
```



The result is accepted after testing. The detail also records that the agent process stopped before a normal completion.

1. Run the command in the code folder and verify eleven successful tests.
2. Replay the menu, error, order, restaurant, and tracking in the browser.
3. Record results and limitations in a verification report.
4. Have the report examined before accepting the result in Swarm.

In this workshop, the trainer added HTTP tests, fixed SQLite connection cleanup, and cleared stale basket errors. The trainer submitted docs/FORMATEUR.md, recorded a gate with file hashes, and accepted the deliverable through public Swarm operations.

This is verification by the trainer in this session, not an independent review. Acceptance does not turn the interrupted execution into a normal completion, and does not configure an autonomous hierarchy.

Expected result. You can justify acceptance using tests, a real journey, and a report, even when the history includes failed attempts.

After confirmation in the current reference, the basket clears and tracking remains available in the tab. The Swarm screenshot above retains historical acceptance; REFONTE.md separately describes verification of the displayed revision.

34 Diagnose Failed to fetch

This message means the browser did not obtain the expected response. In the incident observed during the workshop, the Casa Pizza server had stopped.

1. Check the terminal running app.py. Keep it open throughout the journey.
2. Open `http://127.0.0.1:18841/api/catalog`. If it does not respond, check server and port before the form.
3. Restart from the folder containing app.py with the same SQLite database if you want to retain orders.
4. Refresh the customer page, inspect the basket and confirm once. A new order must display a new number.

```
python3 app.py
curl -i http://127.0.0.1:18841/api/catalog
```

The current reference displays `Seurveur local inaccessible` with a prompt to check the terminal. This message does not restart the server: the learner must restart it.

If the menu responds but ordering fails

Open the browser developer tools, select Network and reproduce once. Find POST `/api/orders`. Record HTTP status, response and time; look for the corresponding server-terminal error. A JSON business refusal, such as the minimum, differs from receiving no network response.

Open the site through the local server, not the HTML file directly. 127.0.0.1 identifies the browser's computer: on another computer it does not refer to the trainer's server. Do not conclude that CORS is responsible without inspecting the request.

To share a diagnosis, copy the message, route, status and steps. Remove tokens, cookies, authentication headers and contact details. Do not send an unreviewed full network export.

35 Repeat and explain the journey

Before finishing, explain to another beginner how you obtained and checked the product.

Exercises and expected answers

1. Does one Reine meet the minimum? Yes: 12.50 euros of products, or 15.40 euros including delivery.
2. Is one Margherita enough? No: 9.50 euros of products, even though the total including delivery reaches 12.40 euros.
3. Does the agent's text prove everything works? No: inspect executed tests and replay the journey.
4. Why use Git here? To prepare a repository and preserve changes. The kit does not push anything to a remote repository.
5. Does the green task mean the agent finished normally? It indicates result acceptance. Read process status and history separately.

Before closing

1. Keep the code, report, and test results.
2. Stop Casa Pizza with Ctrl+C in its terminal.
3. From the Swarm folder, stop only your workshop server.

```
./swarm.sh stop --root "$HOME/projects/pizza-workshop"
```

If you selected 18842, add --address 127.0.0.1:18842. Do not remove the database or history to fix a simple port problem.

For the trainer

The kit includes atelier_swarm.py, reference code, creer_plan_pedagogique.py, and the screenshots used here. It excludes credentials, private Swarm sessions, order databases, and Git history. Historical producer reports are preserved; docs/FORMATEUR.md records final verification and its limitations. The reference application, helper messages, and source reports remain in French.

A next improvement can target one behavior, such as allergen information. Write its criteria, launch a limited task, and repeat verification before acceptance.

Expected result. You can repeat the workshop and explain how a verified delivery differs from code generation alone.

Pause and stop

Mission pause holds new starts; it does not kill already active attempts. Stopping an attempt targets a specific process. Closing a tab does not stop agents. Before closing the workshop, inspect active attempts and choose the appropriate action; stopping the cockpit server does not accept results.

36 Workshop scope and evidence

The reference lets you test a local application. Advanced capabilities described here must be distinguished from those actually executed in the workshop.

Topic	Observed scope
Application	Menu, basket, order, SQLite, tracking and manual restaurant progression
Verification	11 tests on the trainer host and browser journey on desktop and 390 px
Six-task plan	Created to teach graph reading; none of those tasks executed
Policies	Forms and previews inspected; effects not confirmed
Outside workshop	Real payment, logistics, timed automatic progression, publication and production operation
Not demonstrated	Autonomous hierarchical mission, original independent review and complete accessibility audit

Sources for checking the mechanisms

Product reference: Swarm repository, <https://github.com/moOogly/swarm>. Repository documentation: docs/en/USER-GUIDE.md and docs/en/ENGINE-ACCEPTANCE.md. Contract: tools/agent-workflows/CONTRACT.md. Policies and receipts are implemented in validation_policy_config.go and automatic_validation.go.

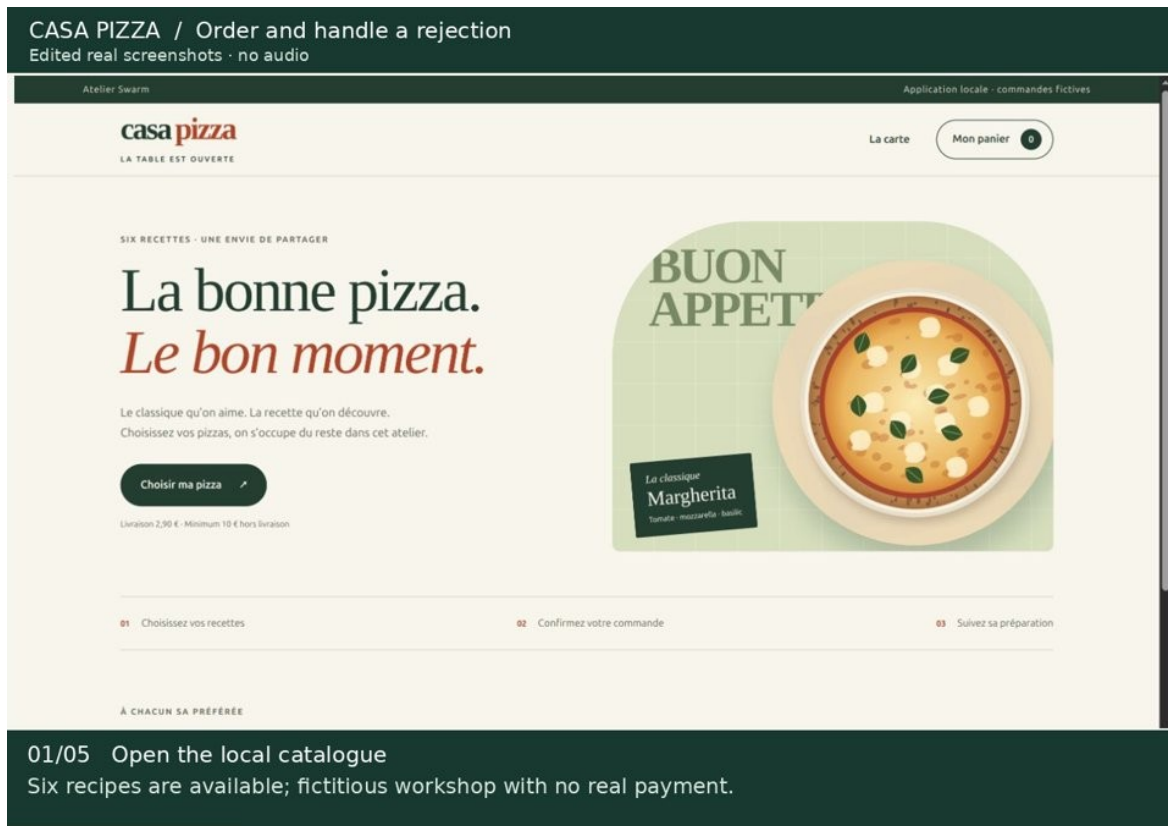
Workshop reference: projet-pizza/docs/FORMATEUR.md, docs/REFONTE.md, tests/test_app.py and tests/test_http.py. The 9 October 2026 screenshots show a development version; labels may change. Test results are not extrapolated to another installation.

The kit excludes keys, private sessions, order databases and Git history. It publishes nothing and installs no external service. Historical producer reports remain French and distinguish unexecuted checks from final verification.

37 Animated tutorials to repeat the journey

This addition preserves the previous 36 chapters. Open the offline reader, choose a module and repeat a step in your workshop before proceeding.

Open the step reader and videos



Captions are burned into the videos. No audio or automatic playback.

From requirements to the graph • 24 s • chapters 8–11 and 17–20

Prepare Swarm checks • 24 s • chapters 21–26

Order and handle a rejection • 30 s • chapters 27–30

From restaurant to customer tracking • 42 s • chapters 31–34

Extract the entire ZIP before opening. MP4 files can also be opened directly in tutoriels/media. GIFs are reduced previews; step mode holds each screenshot on screen and the transcript provides explanations.

38 Answered exercises and evidence scope

Pause, answer, then check. The reader also contains the answers.

1 From requirements to the graph

Which task precedes tests? Does the graph prove execution?

Order and tracking precedes Automatic tests. No: all six tasks are unexecuted, with no active agent. The teaching script created this plan separately from the saved requirements.

2 Prepare Swarm checks

Does a successful command mean the mission is complete?

No. It provides evidence for the files and criteria linked to the candidate. Visual testing, review and acceptance conditions remain distinct. No policy was applied and no task was accepted in this recording.

3 Order and handle a rejection

Why is EUR 12.40 including delivery insufficient for one pizza?

The minimum applies only to pizzas: EUR 9.50 is below EUR 10. The EUR 2.90 delivery charge does not count towards that minimum.

4 From restaurant to customer tracking

Who advanced the order? How can you check it is the same one?

The instructor used the restaurant buttons. Compare order number, contents and total between restaurant and customer. This journey demonstrates neither real delivery nor an autonomous agent.

What was actually recorded

On 9 October 2026, actions were performed in an isolated cockpit and local application. Successive screenshots were edited into still videos and captioned GIFs; these are not continuous screencasts. No AI provider was called for this demonstration.

Requirements were saved. The teaching plan was created separately by the public script, without executing its six tasks. Check policies were previewed and cancelled. The customer flow and restaurant transitions were actually performed using fictitious data. The eleven reference tests passed; this does not prove autonomous mission execution.